PROJECT SYNOPSIS
On

"LexAI – AI-Powered Legal Assistant & Contract Intelligence Platform"

Submitted to

Chhatrapati Shivaji Maharaj University, Panvel, Navi Mumbai

for the partial fulfillment of requirement for the Degree of

MASTER OF COMPUTER APPLICATION

Submitted By

Rohan Bongane
Harsh Jha
Aishwariya Bhunia
Nitu Sharma

Guide

Dr. Surekha Kohle

Department of Computer Science & Information Technology
Chhatrapati Shivaji Maharaj University, Panvel, Navi Mumbai

Academic Year:
2026–27

1. Introduction

The legal domain is characterized by densely packed terminology and lengthy documents, creating significant barriers for individuals and small businesses. Contracts and terms of service are often obfuscated by specialized jargon, making it difficult for non-lawyers to comprehend their rights, obligations, and hidden risks.

Manually reviewing these documents requires immense cognitive effort and frequently necessitates consulting legal professionals. The high cost of legal counsel often leads to contracts being signed without a full understanding of their implications. There is a pressing need for automated tools that can democratize access to basic legal comprehension.

LexAI is an AI-assisted legal information platform designed to bridge this gap. Utilizing a Flask-based web architecture, LexAI allows users to securely upload legal contracts (PDF or DOCX). The platform extracts and processes text to perform semantic analysis, providing executive summaries and identifying critical clauses.

To mitigate the "hallucination" risks associated with generative models, LexAI implements a Retrieval-Augmented Generation (RAG) architecture using the Google Gemini LLM. Grounding responses exclusively in the retrieved text of the user's specific document or a curated Knowledge Base helps reduce the likelihood of inaccurate claims. While LexAI assists in comprehension, it does not replace professional legal advice and acts primarily as an informational tool to aid preliminary review.

2. Literature Review

Jain et al. [1] highlight the complexities of summarizing legal documents, emphasizing the necessity of preserving factual integrity. Traditional extractive methods miss semantic nuance, while abstractive generative models risk fabricating legal precedent. LexAI adopts a deterministic chunking strategy prior to abstraction to generate verifiable summaries.

Chalkidis et al. [2] demonstrated with LEGAL-BERT that generic language models frequently misinterpret legal nuances, necessitating domain-aware processing. LexAI addresses this by combining the efficient all-MiniLM-L6-v2 embedding model with strict JSON-schema prompting via the Gemini LLM to enforce analytical constraints.

Lewis et al. [3] established Retrieval-Augmented Generation (RAG) as a standard for knowledge-intensive NLP tasks, proving that querying an external vector index reduces factual hallucinations. LexAI incorporates this by utilizing ChromaDB as a semantic vector store, ensuring the AI processes text explicitly present in the uploaded document.

Surveys by Ariai et al. [4] and Martinez-Gil [5] underscore challenges in legal QA systems concerning privacy and trust. Public generative AI interfaces pose security risks for confidential contracts, and unconstrained models may hallucinate standard practices instead of citing negotiated terms.

Research Gap

While existing literature supports RAG in controlled settings, there is a gap in accessible systems that combine private document analysis with a securely isolated environment. LexAI addresses this by enforcing database-level filtering (document ID isolation), ensuring users only query their private contracts safely.

3. Aims & Objective

The primary aim of LexAI is to develop a secure, accessible web-based platform that utilizes Artificial Intelligence and RAG to demystify complex legal documents.

Implemented Objectives:

- **User Authentication & Security:** Develop secure registration and session management using Flask-Login and Bcrypt.
- **Document Processing:** Enable secure upload and raw text extraction from PDF/DOCX using PyMuPDF and python-docx.
- **AI Legal Analysis:** Engineer JSON prompts that constrain the Gemini API to generate executive summaries and highlight key clauses.
- **RAG-Based Retrieval:** Implement a vector embedding pipeline using SentenceTransformers and ChromaDB.
- **Secure Isolation:** Enforce database filtering so users can only access their own private documents.
- **Document Chatbot:** Create a conversational interface for users to query their specific uploaded contracts.
- **Knowledge Base:** Provide an admin-curated Knowledge Base for general legal queries.
- **Deterministic Fallback:** Implement a robust parser to handle external API failures by formatting local content.

Future Objectives (Planned):

- **Optical Character Recognition (OCR):** Process scanned, image-based legal documents lacking embedded text.
- **Cloud Migration:** Migrate local ChromaDB and file storage to persistent cloud infrastructure.

4. Problem Definition

~~The legal ecosystem remains largely inaccessible to the general public and~~ small enterprises due to the volume and complexity of legal text. When an individual faces a standard contract, they encounter several immediate challenges:

Manual Review Effort

Contracts are often filled with repetitive boilerplate text and complex formatting. Reading these documents requires significant time. For many, this effort leads to signing documents without thorough review.

Obfuscated Clauses and Hidden Risks

Critical obligations, termination conditions, and liability waivers are frequently buried within archaic terminology. Identifying these risks without training is highly error-prone, carrying potential financial implications.

Information Retrieval Friction

Finding specific clauses (e.g., late payment penalties) in a massive document is tedious. Traditional keyword search often fails if the user does not guess the exact terminology used by the drafting lawyer.

Unreliable Public AI Solutions

While public generative AI chatbots are available, they provide generic, ungrounded responses. Furthermore, pasting confidential contracts into public interfaces poses severe privacy and compliance risks.

5. Proposed Model

LexAI addresses the aforementioned problems by proposing a secure, decoupled client-server architecture. This model ensures that documents are processed locally into a vector database, and an LLM is strictly constrained via RAG to answer questions based only on the user's uploaded text or the curated Knowledge Base.

5.1 System Architecture Overview

The system follows a modular architecture separating the user interface, backend orchestration, data storage, and external AI services. The data flow proceeds as follows:

User ➔ Web Interface ➔ Flask Backend ➔ Authentication and Document Processing ➔ MySQL / Local File Storage / ChromaDB ➔ Gemini API ➔ Response to User

5.2 Frontend Application

The presentation layer is built utilizing HTML5, CSS3, and JavaScript, with Bootstrap 5 ensuring a responsive layout across devices. It handles user authentication forms, document uploads, and asynchronous chat interactions. To securely render AI-generated Markdown responses, the frontend employs marked.js for formatting and DOMPurify to sanitize outputs against Cross-Site Scripting (XSS) vulnerabilities.

5.3 Backend Processing Layer

A Python-based Flask application serves as the core backend orchestration layer. It is responsible for routing HTTP requests, managing user sessions, processing uploaded documents, executing retrieval logic, and handling communication with the external Gemini API. This layer acts as a secure intermediary, ensuring that raw data is processed before any AI integration occurs.

5.4 Relational Database and File Storage

MySQL, managed via Flask-SQLAlchemy, acts as the primary relational database. It stores securely hashed user credentials, document metadata (such as file paths and upload timestamps), generated executive summaries, and chat conversation histories. Uploaded PDF and DOCX files are stored directly on the local server file system in isolated, user-specific directories.

5.5 Semantic Vector Storage

To enable semantic search, extracted document text is processed by a local SentenceTransformers model (all-MiniLM-L6-v2) to generate mathematical embeddings. These vectors, along with document chunk metadata, are indexed in ChromaDB, an open-source embedding database optimized for local RAG pipelines.

5.6 AI Analysis and RAG Integration

When a query is made, the retrieved text chunks from ChromaDB are supplied to the Google Gemini API. The API is constrained by structured system prompts to generate contextual answers or executive summaries grounded solely in the provided text. The architecture strictly separates Document-Specific Chat (querying private uploads) from the General Legal Knowledge Base (querying an admin-curated repository).

5.7 Security and Reliability

Security is enforced at multiple levels. Authentication relies on Flask-Login and Bcrypt password hashing. ChromaDB queries are explicitly filtered by user ID and document ID to prevent cross-contamination of private data. While these mechanisms aim to minimize unauthorized access and reduce AI hallucinations, the system relies on a deterministic fallback parser to maintain baseline functionality if the external Gemini API becomes temporarily unavailable.

6. Methodology

The operational methodology of LexAI is structured into a chronological processing pipeline, transforming raw document uploads into interactive, semantically searchable data.

6.1 User Registration and Authentication

Users initiate their session by registering an account. Passwords are mathematically salted and hashed using the Bcrypt algorithm before database insertion. The Flask-Login extension manages secure, HTTP-only session cookies, verifying user identity during subsequent requests and preventing unauthorized access to protected routes.

6.2 Document Upload and Validation

Authenticated users upload legal contracts in PDF or DOCX formats via a secure multipart form. The backend strictly validates the file extension and MIME type to reject malicious payloads. Validated files are saved to the local filesystem within a designated upload directory.

6.3 Text Extraction

The system invokes PyMuPDF to parse PDF files and python-docx for Word documents. These libraries extract pure unicode text while discarding visual formatting. Currently, the methodology relies on embedded text layers; scanned, image-based PDFs cannot be fully processed as Optical Character Recognition (OCR) is not yet implemented.

6.4 Text Preprocessing and Chunking

Because Large Language Models are constrained by token limits, the extracted text is divided into smaller, manageable chunks. To ensure that continuous legal clauses are not arbitrarily split, the algorithm utilizes overlapping windows (e.g., a short overlap between consecutive chunks). This preserves the semantic context across chunk boundaries.

6.5 Embedding Generation and Vector Indexing

Each text chunk is passed through the local all-MiniLM-L6-v2 SentenceTransformers model to generate a high-dimensional mathematical vector. These vectors are ingested into ChromaDB. Crucially, each vector is appended with metadata containing the specific document ID and user ID to ensure strict indexing.

6.6 Semantic Retrieval

During a chat interaction, the user's natural language question is converted into an embedding using the identical SentenceTransformers model. ChromaDB performs a cosine similarity search between the question vector and the indexed document chunks. The retrieval is strictly filtered by the authenticated user's ID, returning only the most relevant chunks.

6.7 AI-Based Document Analysis

For automated analysis, the retrieved document content is supplied to the Gemini API alongside a restrictive JSON-schema prompt. The prompt instructs the model to extract key clauses, summarize obligations, and identify potential risks based exclusively on the provided text, returning a structured summary for the user dashboard.

6.8 Document-Specific Chat and General Legal Chat

The methodology employs two distinct retrieval paths. The Document-Specific Chat filters vector searches exclusively against the user's private uploaded file. Conversely, the General Legal Chat bypasses user documents entirely, querying a separate ChromaDB collection populated only with an administrator-curated Knowledge Base, ensuring private data is never mixed with general inquiries.

6.9 Fallback and Error Handling

In the event of a Gemini API outage (e.g., quota limits or network errors), a deterministic Python fallback mechanism is triggered for Knowledge Base queries. This parser bypasses the LLM entirely, retrieving the most relevant raw text chunks from ChromaDB, formatting them into Markdown, and presenting them directly to the user to maintain system availability.

6.10 Testing and Validation

System reliability is verified using the Pytest framework. The testing methodology includes unit tests for the authentication flows, ensuring registration validations and login route protections function correctly. Database isolation tests verify that users cannot access records or vectors belonging to other accounts, utilizing separate test databases to prevent corruption of production data.

---

7. Plan of Work

The development of LexAI was executed in sequential phases, prioritizing core infrastructure and security before integrating advanced artificial intelligence components. The structured timeline and outcomes are detailed below.

Phase	Development Activity	Tools / Technologies	Status	Expected Outcome
Phase 1	Project setup and authentication.	Flask, MySQL, Bcrypt, Flask-Login	Implemented	Secure user accounts and isolated sessions.
Phase 2	Document upload and text extraction.	PyMuPDF, python-docx, werkzeug	Implemented	Reliable ingestion of raw text from PDF and DOCX files.
Phase 3	AI-powered legal document analysis.	Google Gemini API, JSON-Schema	Implemented	Automated generation of structured executive summaries.
Phase 4	RAG pipeline and document chat.	SentenceTransformers, ChromaDB, AJAX	Implemented	Semantic search enabling context-aware chat interaction.
Phase 5	Knowledge Base, fallback mechanism, and testing.	Flask, Pytest, Python Parsers	Implemented	Curated legal chat and resilient external API error handling.
Phase 6	OCR support for scanned documents.	Tesseract OCR, Vision APIs	Future Scope	Ability to analyze image-based PDFs lacking text layers.
Phase 7	Multilingual support or cloud deployment.	Translation Pipelines, AWS S3	Future Scope	Scalable enterprise deployment and broader language accessibility.

These phases build logically upon one another; foundational authentication and file handling (Phases 1 and 2) were strict prerequisites for the RAG

pipeline (Phases 3 and 4). Future enhancements, such as OCR and cloud migration (Phases 6 and 7), will extend the system's capabilities without disrupting the core architecture established in the implemented phases.

8. Conclusion

The LexAI project successfully addresses the significant accessibility barrier posed by dense legal terminology and lengthy contracts. By developing a secure, web-based platform that automates the extraction and semantic analysis of legal documents, the system provides individuals and small enterprises with a practical tool for preliminary contract review. The integration of local vector indexing with Retrieval-Augmented Generation (RAG) ensures that the artificial intelligence is firmly grounded in the user's actual document, aiming to minimize the likelihood of hallucinatory or inaccurate responses.

A critical achievement of the project is the robust implementation of two distinct conversational interfaces. The Document-Specific Chat allows users to interrogate their private uploads securely, enforced by strict database-level filtering that isolates user data. Simultaneously, the General Legal Knowledge Base provides access to curated legal concepts without cross-contaminating private records. Furthermore, the inclusion of a deterministic fallback mechanism ensures that the platform maintains baseline functionality and provides uninterrupted access to its curated repository even during external API outages.

Despite these advancements, LexAI possesses acknowledged limitations. The current text extraction methodology relies on embedded text layers; therefore, scanned or image-based documents cannot be processed, as Optical Character Recognition (OCR) is not yet implemented. Additionally, the system currently lacks multilingual capabilities, and while RAG significantly reduces factual errors, it does not completely eliminate the risk of AI inaccuracy. LexAI is explicitly designed as an informational assistant and does not replace the necessity for qualified legal professionals.

Future improvements will focus on expanding the platform's accessibility and scalability. The planned integration of OCR will allow for the processing of legacy scanned contracts, while multilingual support would democratize legal comprehension across broader demographics. Ultimately, LexAI represents a functional, verifiable prototype that makes significant practical contributions toward rendering complex legal information transparent and accessible.

---

9. References

[1] Jain, D., Borah, M. D., & Biswas, A. (2021). "Summarization of legal documents: Where are we now and the way forward." **Computer Science Review**, 40, 100388.

[2] Chalkidis, I., Fergadiotis, M., Malakasiotis, P., Aletras, N., & Androutsopoulos, I. (2020). "LEGAL-BERT: The Muppets straight out of Law School." **Findings of the Association for Computational Linguistics: EMNLP 2020**.

[3] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Riedel, S. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." **Advances in Neural Information Processing Systems (NeurIPS)**.

[4] Ariai, F., Mackenzie, J., & Demartini, G. (2025). "Natural Language Processing for the Legal Domain: A Survey of Tasks, Datasets, Models, and Challenges." **ACM Computing Surveys**, 58(6).

[5] Martinez-Gil, J. (2023). "A survey on legal question–answering systems." **Computer Science Review**, 48, 100551.

Prof. Dr. Surekha Kohle	**Dr. Praveen Gupta**
Guide	H.O.D.
CS/IT Department	CS/IT Department